

BUILDING A MULTI-AGENT BUSINESS IN CLAUDE

A complete implementation guide for specialized agents, controlled handoffs,
software delivery, and business operations

Includes copy-ready Claude Code subagent files, permissions, workflow gates, testing, rollout, and governance

Version 1.0 | July 25, 2026

Prepared as a practical operating manual. Product features and model availability may change; verify against the official Anthropic sources listed at the end.

How to Use This Guide

Recommended path

Use Claude Code custom subagents for a controlled virtual company. Start with the 14-agent core team, run agents sequentially, keep independent reviewers read-only, and require human approval before architecture changes and production releases.

This guide gives you three implementation levels:

Level	Best for	What you build
1. Claude Projects	No-code experimentation and role-specific chats	Separate projects with instructions and knowledge; handoffs are manual.
2. Claude Code subagents	A real controlled multi-agent workflow inside a repository	Markdown agent definitions, tool restrictions, permissions, memory, and automatic or explicit delegation.
3. Agent SDK / agent teams	Production automation or large coordinated workloads	Programmatic agents, external integrations, audit controls, and optional parallel execution.

The recommended sequence is to prove the workflow with subagents before building a fully automated SDK system.

Contents

- 1. Decide which Claude agent approach to use
- 2. Design principles for reliable specialized agents
- 3. Complete business and software agent catalog
- 4. End-to-end controlled business workflow
- 5. Install and prepare Claude Code
- 6. Create the folder structure and project memory
- 7. Build each subagent file
- 8. Configure permissions, safety, and tool access
- 9. Create the Chief of Staff orchestration layer
- 10. Run, test, and improve the agent organization
- 11. Scale to Agent SDK, integrations, and agent teams
- 12. Copy-ready core agent definitions
- 13. Additional business agent blueprints
- 14. Rollout checklist and troubleshooting
- 15. Official sources

1. Decide Which Claude Agent Approach to Use

1.1 Claude Projects: easiest, but mostly manual

Claude Projects are self-contained workspaces with project instructions, knowledge, and focused chats. They are useful for creating separate role-based workspaces, such as Architect, Business Analyst, and Marketing Manager. However, one project does not automatically execute a controlled handoff to another project. [1][2]

Strength	Limitation
No coding required	Handoffs normally require manual copying or a shared document
Each project has separate instructions and knowledge	Not a strong orchestration system for software changes
Useful for role experimentation	Permissions and tools are not as granular as Claude Code subagents
Good for business research and writing	Harder to enforce gates and independent review

1.2 Claude Code custom subagents: recommended

Claude Code custom subagents are specialized assistants stored as Markdown files with YAML frontmatter. Each agent can have a separate system prompt, context window, tool allowlist, model, permission mode, skills, memory, hooks, and other controls. Claude uses the agent description to decide when to delegate, and you can also invoke agents explicitly. [3]

Why this matches your goal

You want each specialist to do one thing at a time. Subagents let you make the Architect read-only, allow the Code Writer to edit files, prevent the PR Reviewer from changing code, and require structured handoffs between roles.

1.3 Agent teams and Agent SDK: advanced options

Agent teams coordinate multiple Claude Code sessions with shared tasks and inter-agent messaging, but the feature is experimental and disabled by default. It is intended for coordinated parallel work, not as the first choice for a strict one-at-a-time workflow. [4]

The Claude Agent SDK lets you build production agents in Python or TypeScript using the same agent loop and tools as Claude Code. Use it after the manual subagent workflow is stable and you need repeatable automation, APIs, schedules, external events, or audit reporting. [5]

Choose this	When
Claude Projects	You want to learn and manually test role prompts without coding.
Claude Code subagents	You want controlled role separation, repository access, handoffs, and software delivery.
Agent SDK	You need a repeatable production service or integration with your own application.
Agent teams	You need several independent sessions to communicate and work in parallel, and you accept experimental limitations.

2. Design Principles for Reliable Specialized Agents

One job per agent

Every agent should own one decision or deliverable, not an entire department.

Why: Prevents overlapping answers and unclear accountability.

Explicit inputs

State exactly what the agent is allowed to receive: approved requirements, architecture, code diff, test results, or business data.

Why: Keeps agents from inventing missing context.

Fixed output contract

Require a named deliverable and a standard handoff format.

Why: Makes the next agent easy to trigger and audit.

Limited tools

Give reviewers read-only tools; give editing and deployment tools only to roles that need them.

Why: Prompt instructions alone do not enforce permissions; tool rules do. [6]

Independent approval

No agent may approve its own work.

Why: Separates execution from verification.

Human gates

Humans approve business commitment, material architecture changes, legal risk, spending, and production deployment.

Why: Preserves accountability for high-impact decisions.

Sequential by default

Run one agent at a time until the workflow is trustworthy.

Why: Reduces conflicting edits, cost, and coordination failures.

Evidence over confidence

Require file references, test output, assumptions, and unresolved questions.

Why: Prevents polished but unsupported conclusions.

Stop conditions

Every agent must stop when its defined output is complete or a blocker is found.

Why: Prevents scope drift.

Least privilege

Default to read-only, then add the minimum required capabilities.

Why: Limits accidental or unauthorized changes.

The standard agent contract

Agent name:
Single responsibility:
Use when:
Required inputs:

Allowed tools:
Disallowed actions:
Required deliverable:
Acceptance criteria:
Stop condition:
Recommended next agent:
Human approval required:

The universal handoff

Task completed
Inputs reviewed
Decisions made
Deliverables produced
Files or systems affected
Assumptions
Risks
Open questions
Acceptance criteria status
Recommended next agent
Human approval required: Yes or No

3. Complete Agent Catalog

3.1 Core 14-agent team

Agent	Single responsibility	Default access
Chief of Staff	Routes work, enforces sequence, checks handoffs	Read-only
Program Manager	Creates work plan, dependencies, timeline, risk log	Read-only
Market Researcher	Validates demand, customers, competitors, evidence	Read and web research
Business Analyst	Converts business needs into testable requirements	Read-only
Product Manager	Defines MVP, priorities, roadmap, success metrics	Read-only
UX/UI Designer	Creates user flows, interface rules, accessibility needs	Read-only or design tools
Solution Architect	Designs components, APIs, data flow, tradeoffs	Read-only / plan
Security Architect	Creates threat model and security requirements	Read-only / plan
Codebase Researcher	Maps existing code and identifies change points	Read-only
Code Writer	Implements only approved scope and tests	Read, edit, write, shell
QA Engineer	Executes independent functional and regression testing	Read and shell
PR Reviewer	Reviews correctness, maintainability, security, and tests	Read-only and shell
DevOps Engineer	Builds CI/CD and approved deployment automation	Controlled write and shell
Release Manager	Verifies readiness and recommends release or hold	Read-only

3.2 Business growth agents

Agent	Single responsibility
Business Strategist	Business model, differentiation, go/no-go recommendation
Financial Analyst	Budget, forecast, break-even, unit economics, scenario analysis
Privacy & Compliance	Data handling, retention, consent, regulatory review flags
Legal Risk Reviewer	Contract and policy risks requiring qualified legal review
Operations Manager	Standard operating procedures, checklists, escalation paths
Marketing Strategist	Positioning, campaigns, channels, acquisition metrics
Content Creator	Approved website, social, email, and educational content
Sales Strategist	Qualification, discovery, proposals, objections, pipeline process
Customer Onboarding	Implementation plan, training, adoption milestones
Customer Support	Issue classification, verified solutions, escalation
Customer Success	Retention, account health, renewals, customer outcomes
Data / BI Analyst	Dashboards, conversion, retention, revenue, product usage
Knowledge Manager	Source of truth, decisions, documentation freshness

Agent	Single responsibility
AI Cost Controller	Token use, model routing, workflow cost and limits

3.3 Advanced technology and operations agents

Agent	Single responsibility
Data Architect	Schemas, data ownership, migration, quality, retention
Integration Engineer	APIs, third-party systems, MCP connections, error contracts
Test Designer	Test strategy and cases before implementation
Security Reviewer	Independent review of the finished implementation
Site Reliability Engineer	Monitoring, alerts, service objectives, capacity, recovery
Incident Commander	Coordinates production incidents and post-incident learning
Technical Writer	User, operator, API, architecture, and release documentation
Vendor & Procurement	Vendor comparison, renewal tracking, dependency and contract risk
HR & Recruiting	Job descriptions, interview rubrics, onboarding and training

Do not create every role on day one

Start with the core 14. Add a specialist only when a recurring task cannot be handled safely or clearly by an existing agent. Too many agents increase cost, duplicated work, and handoff complexity.

4. End-to-End Controlled Business Workflow

1	Idea intake Owner: Chief of Staff Purpose: Clarify the request and choose the workflow Required output: Work request and routing decision Gate: No development begins
2	Market validation Owner: Market Researcher Purpose: Find evidence of demand and alternatives Required output: Market evidence package Gate: Human go/no-go for further analysis
3	Business feasibility Owner: Business Strategist + Financial Analyst Purpose: Evaluate model, costs, risks, and economic value Required output: Business case and scenarios Gate: Human investment decision
4	Requirements Owner: Business Analyst Purpose: Define users, rules, scope, constraints, and acceptance criteria Required output: Approved requirements package Gate: Requirements approval
5	Product scope Owner: Product Manager Purpose: Define MVP, priorities, roadmap, and success metrics Required output: Product brief and prioritized backlog Gate: Scope approval
6	Experience design Owner: UX/UI Designer Purpose: Define user journeys, screens, behaviors, and accessibility Required output: UX specification Gate: Design approval for major user flows
7	Architecture Owner: Solution Architect Purpose: Create technical design and implementation sequence Required output: Architecture package Gate: Human approval for material changes

8	Security design Owner: Security Architect Purpose: Threat model and security acceptance criteria Required output: Security design review Gate: No unresolved blocker
9	Code research Owner: Codebase Researcher Purpose: Locate current behavior and safe change points Required output: Codebase map and change plan Gate: Implementation scope confirmed
10	Implementation Owner: Code Writer Purpose: Make the smallest approved change and add tests Required output: Code changes and local validation Gate: Must go to independent QA
11	Quality assurance Owner: QA Engineer Purpose: Run acceptance, regression, negative, and integration tests Required output: QA report and defects Gate: All blockers resolved
12	Independent review Owner: Security Reviewer + PR Reviewer Purpose: Review final diff and evidence Required output: Review decision and findings Gate: Approval or request changes
13	Delivery preparation Owner: DevOps Engineer Purpose: Prepare CI/CD, environment, migration, monitoring, rollback Required output: Release package Gate: No direct unapproved production action
14	Release decision Owner: Release Manager Purpose: Verify every gate and recommend release or hold Required output: Release readiness report Gate: Human production approval

15

Operations and learning

Owner: SRE + Support + Success + Data Analyst

Purpose: Operate, support, measure, and feed learning back to Product

Required output: Operational metrics and next-iteration input

Gate: Continuous improvement

Change-loop rule**When a downstream agent finds a defect**

Return the work to the agent that owns the defective deliverable. Example: QA sends a code defect to Code Writer; PR Reviewer sends requirement ambiguity to Business Analyst or Product Manager; Security Reviewer sends a design flaw to Security Architect or Solution Architect. Do not let the reviewer silently fix the work it is reviewing.

5. Install and Prepare Claude Code

Anthropic currently documents native installers for macOS, Linux, WSL, and Windows PowerShell. After installation, verify the CLI, sign in, enter the repository, and create a project-level CLAUDE.md. [7][8]

5.1 Install

macOS, Linux, or WSL:

```
curl -fsSL https://claude.ai/install.sh | bash
```

Windows PowerShell:

```
irm https://claude.ai/install.ps1 | iex
```

Alternative Homebrew installation:

```
brew install --cask claude-code
```

Verify and start:

```
claude --version  
cd path/to/your/project  
claude
```

5.2 Authenticate

- Most users: run claude, then use /login and approve the browser request.
- API usage: set ANTHROPIC_API_KEY in your shell before starting Claude Code.
- Enterprise cloud providers: configure Bedrock, Google Cloud, or the organization-approved provider according to your administrator.

5.3 Initialize project context

```
/init
```

Claude Code can generate a CLAUDE.md that documents build commands, test commands, architecture, conventions, and project constraints. Claude automatically reads relevant CLAUDE.md files. Keep the project-level file concise and accurate, and commit it to version control for the team. [8]

Important

The first time you create a new .claude/agents directory during a running session, restart Claude Code if the new agent is not detected. Existing watched directories update automatically when agent files change. [3]

6. Create the Folder Structure and Project Memory

6.1 Recommended repository structure

```

your-business-project/
├── CLAUDE.md
├── .claude/
│   ├── agents/
│   │   ├── chief-of-staff.md
│   │   ├── program-manager.md
│   │   ├── market-researcher.md
│   │   ├── business-analyst.md
│   │   ├── product-manager.md
│   │   ├── ux-designer.md
│   │   ├── solution-architect.md
│   │   ├── security-architect.md
│   │   ├── codebase-researcher.md
│   │   ├── code-writer.md
│   │   ├── qa-engineer.md
│   │   ├── pr-reviewer.md
│   │   ├── devops-engineer.md
│   │   └── release-manager.md
│   ├── settings.json
│   └── settings.local.json
├── docs/
│   ├── requirements/
│   ├── architecture/
│   ├── decisions/
│   ├── test-reports/
│   ├── release-notes/
│   └── operations/
└── src/

```

6.2 Project-level versus personal agents

Location	Scope	Use
.claude/agents/	Current project	Business- or repository-specific agents. Commit these to Git when the team should share them.
~/.claude/agents/	All your projects	Personal reusable agents, such as a general writing reviewer.
--agents CLI flag	Current session only	Temporary testing or automation without saving files.
Managed settings / plugins	Organization or plugin scope	Central administration and distribution.

6.3 CLAUDE.md starter template

```

# Project Mission
Describe the customer problem, product, and business objective in 3-5 sentences.

# Architecture Summary
- Major components
- How data moves between them
- External dependencies

# Commands
- Install: <command>
- Build: <command>
- Test: <command>
- Lint: <command>
- Run locally: <command>

# Engineering Conventions
- Language and framework rules
- Naming and folder conventions

```

- Error-handling pattern
- Logging and observability standards
- Test expectations

Hard Constraints

- Never edit generated files.
- Never use production credentials locally.
- Every API route requires approved authentication and authorization.
- Database schema changes require an approved migration and rollback plan.
- No agent may merge or deploy without human approval.

Agent Workflow

Use project subagents in this order unless the Chief of Staff documents a reason:
Program Manager -> Business Analyst -> Product Manager -> UX Designer ->
Solution Architect -> Security Architect -> Codebase Researcher -> Code Writer ->
QA Engineer -> PR Reviewer -> DevOps Engineer -> Release Manager.

Required Handoff

Every agent must return the universal handoff defined in this guide.

Known Gotchas

List recurring project-specific mistakes and how to avoid them.

7. Build Each Subagent File

A custom subagent file contains YAML frontmatter followed by a Markdown system prompt. Only name and description are required, but useful controls include tools, disallowedTools, model, permissionMode, maxTurns, skills, MCP servers, hooks, memory, effort, background behavior, worktree isolation, and display color. [3]

7.1 Minimal valid agent

```
---
name: code-reviewer
description: Reviews completed code changes for correctness, security, and maintainability. Use after
implementation and testing.
tools: Read, Grep, Glob
model: sonnet
permissionMode: plan
---
```

You are an independent code reviewer.
Review the completed changes and return findings with severity, evidence, impact, and recommendation.
Do not edit files. Stop after the review report.

7.2 Frontmatter fields you are most likely to use

Field	Purpose
name	Required unique lowercase identifier using letters and hyphens.
description	Required. Explain exactly when Claude should delegate to this agent.
tools	Allowlist of tools available to the agent. Omit only when broad access is intentional.
disallowedTools	Explicitly removes tools from the inherited or specified tool set.
model	Use an alias such as haiku, sonnet, opus, or inherit, subject to availability.
permissionMode	Common choices: plan for read-only planning/review; default for controlled editing.
maxTurns	Caps the number of agentic turns and limits runaway work.
memory	user, project, or local persistent memory scope for cross-session learning.
skills	Preloads named skills into the agent context.
mcpServers	Restricts the external MCP servers available to this agent.
hooks	Adds lifecycle controls for the specific agent.
effort	Overrides reasoning effort for that agent when supported.
background	Forces background execution when true. Leave unset for strict sequential control.
isolation	Use worktree for an isolated Git checkout when parallel editing is necessary.

7.3 How to write a strong description

Weak	Strong
Helps with architecture	Creates a technical architecture from approved requirements. Use after product scope is approved and before implementation.

Weak	Strong
Reviews code	Independently reviews completed code changes after tests pass. Never edits files or approves its own work.
Manages projects	Creates the delivery plan, dependencies, owners, risks, and status; does not define product requirements or write code.

Delegation depends on descriptions

Claude uses the description to decide when an agent is appropriate. Include the trigger, the deliverable, and the boundary between this role and neighboring roles. [3]

8. Configure Permissions, Safety, and Tool Access

Claude Code permissions are enforced by the application rather than by prompt wording. Use permission rules, modes, and hooks to control actual capabilities. [6]

8.1 Recommended tool matrix

Role type	Typical tools	Hard boundary
Chief of Staff	Read, Glob, Grep	No editing or deployment
Program / Business / Product	Read, Glob, Grep	No source edits
Architects and reviewers	Read, Glob, Grep; optional read-only Bash	No Write or Edit
Market Researcher	Read, Glob, Grep, WebSearch, WebFetch	No edits to product code
Codebase Researcher	Read, Glob, Grep, read-only Bash	No Write or Edit
Code Writer	Read, Glob, Grep, Write, Edit, Bash	No merge or production deployment
QA Engineer	Read, Glob, Grep, Bash	Do not change tests to hide failures
DevOps Engineer	Read, Glob, Grep, Write, Edit, Bash	Production commands require explicit human approval
Release Manager	Read, Glob, Grep, Bash	No release execution

8.2 Permission modes

Mode	Use
plan	Read and explore without editing source files. Best for architects and reviewers.
default / manual	Prompts for tool permissions as needed. Best general mode for controlled work.
acceptEdits	Automatically accepts edits in the approved working area. Use only after the workflow is trusted.
dontAsk	Denies tools unless they are pre-approved. Useful for tightly controlled headless agents.
auto	Uses background safety checks to approve aligned actions; validate carefully before relying on it.
bypassPermissions	Skips most prompts and gives broad access. Avoid for normal business workflows.

Security rule

Never use `bypassPermissions` for an agent that has production credentials, destructive infrastructure access, customer data, or broad filesystem access. Prefer explicit allow and deny rules plus human approval.

8.3 Example project settings

```
{
  "permissions": {
    "deny": [
      "Bash(rm -rf *)",
      "Bash(git push --force*)",
      "Bash(*production*)",
      "Read(.env*)",
      "Read(**/secrets/**)"
    ],
  },
}
```



```
"ask": [  
  "Bash(git push*)",  
  "Bash(terraform apply*)",  
  "Bash(kubectl apply*)",  
  "Bash(*deploy*)"  
]  
}
```

Treat the command patterns above as a starter only. Adapt them to your operating system, toolchain, repository, and actual deployment commands. Test rules in a non-production environment.

8.4 Human approval gates

- Business investment or customer commitment
- Material architecture, data model, or integration change
- Privacy, compliance, legal, or contractual decision
- Spending above an approved threshold
- Production database migration
- Security exception or unresolved high-severity finding
- Production release, rollback, or destructive infrastructure action

9. Create the Chief of Staff Orchestration Layer

The Chief of Staff is responsible for routing and control, not for doing specialist work. It should inspect the request, select the next agent, validate the previous handoff, and stop at approval gates.

9.1 Master orchestration prompt

```
Act as the Chief of Staff for this organization.

Use project subagents and run them strictly one at a time.
Never perform a specialist's deliverable yourself when the specialist exists.
Never start the next agent until the current agent returns the universal handoff.

Default workflow:
1. program-manager
2. market-researcher when the opportunity is unvalidated
3. business-analyst
4. product-manager
5. ux-designer when user-facing behavior changes
6. solution-architect
7. security-architect
8. codebase-researcher
9. code-writer
10. qa-engineer
11. pr-reviewer
12. devops-engineer when delivery configuration changes
13. release-manager

Rules:
- If an agent reports a blocker, stop and report it.
- Do not let an agent approve its own work.
- Do not run agents in parallel unless I explicitly authorize it.
- Require human approval before material architecture changes and production release.
- Preserve every agent's assumptions, risks, and unresolved questions.
- Return work to the owning agent when a reviewer finds a defect.
- Do not allow scope expansion without Product Manager and human approval.

At the end, provide:
- Current phase
- Completed gates
- Blockers
- Pending approvals
- Recommended next action
```

9.2 Routing rules

Request type	Route
New business idea	Market Researcher -> Business Strategist -> Financial Analyst
Unclear requested feature	Business Analyst -> Product Manager
Major user-flow change	UX Designer -> Solution Architect
New service or integration	Solution Architect -> Security Architect -> Integration Engineer
Existing-code modification	Codebase Researcher -> Code Writer
Test failure	QA Engineer or Debugger -> Code Writer -> QA Engineer
Completed pull request	PR Reviewer and Security Reviewer
Release candidate	DevOps Engineer -> Release Manager -> human approval
Production incident	Incident Commander -> relevant specialists -> SRE -> post-incident review

9.3 Sequential execution instruction

Run the workflow in the foreground and strictly one agent at a time.
Do not use background execution.
Do not start the next agent until the current result is required and reviewed.
When invoking an agent, include only the approved input package for that stage.

Current behavior to understand

Recent Claude Code versions may run subagents in the background by default when Claude decides it can. A strong sequential instruction and explicit foreground invocation are important for your workflow. The background field should remain unset rather than forced true. [3]

10. Run, Test, and Improve the Agent Organization

10.1 First dry run

Use the controlled agent workflow for this request:

Create a customer complaint management application where employees can register complaints, attach documents, assign an owner, track status, record resolution notes, and produce management reports.

Begin with program-manager. Run one agent at a time. Stop after Business Analyst and Product Manager complete their deliverables so I can review the scope.

10.2 Test each agent separately

Test	Expected behavior
Boundary test	Ask the Architect to write code. It should refuse and return architecture only.
Permission test	Ask the PR Reviewer to edit a file. The tool configuration should prevent editing.
Missing-input test	Invoke Code Writer without approved requirements. It should stop and name the missing gate.
Handoff test	Verify every required handoff field is present.
Reviewer independence test	Create a deliberate defect and confirm QA or PR Reviewer reports it rather than hiding it.
Scope test	Ask for an unrelated feature during implementation. Code Writer should flag scope expansion.
Safety test	Try a destructive or production command in a non-production sandbox and verify rules block or ask.
Cost test	Run the same task with different model assignments and compare result quality and usage.

10.3 Quality scorecard

Dimension	Question	Score
Role fidelity	Did the agent stay within its single responsibility?	0-5
Input discipline	Did it use approved inputs and identify missing information?	0-5
Evidence	Did it cite files, tests, sources, or calculations?	0-5
Deliverable quality	Is the required output complete and usable?	0-5
Handoff quality	Can the next agent act without reinterpreting the work?	0-5
Safety	Did it respect tools, permissions, and human gates?	0-5
Efficiency	Did it avoid unnecessary steps and model usage?	0-5

Do not automate a workflow until it repeatedly scores well on role fidelity, safety, and handoff quality.

10.4 Improve prompts based on failures

Failure	Prompt or configuration fix
Agent does another role's work	Add explicit exclusions and a stop condition.
Claude does not select the right agent	Rewrite the description with a clear trigger and neighboring-role boundary.
Reviewer edits code	Remove Write and Edit tools; use plan permission mode.
Agent runs too long	Add maxTurns and a precise deliverable.
Context is missing	Add the needed information to CLAUDE.md, project docs, or the invocation prompt.
Repeated outdated decisions	Use a Knowledge Manager and mark superseded documents clearly.
Costs are too high	Route simple work to a faster model and reduce unnecessary agent passes.

11. Scale to Agent SDK, Integrations, and Agent Teams

11.1 When to use the Agent SDK

- You need an API or internal application to trigger workflows.
- You need repeatable runs with structured logs and results.
- You need schedules, webhooks, queues, or event-based automation.
- You need integration with CRM, ticketing, GitHub, Slack, databases, or business systems.
- You need centralized cost controls, approvals, and audit reporting.

The Agent SDK supports production agents in Python and TypeScript with built-in tools and filesystem-based Claude configuration. [5]

11.2 When to use MCP

Model Context Protocol connections expose approved external tools and data to Claude. Scope MCP servers to the agents that need them, such as giving the Sales Agent access to CRM while withholding it from Code Writer. Do not give all agents every connector.

Agent	Possible integration	Restriction
Sales Strategist	CRM	Read leads; require approval before sending customer messages
Customer Support	Ticketing and knowledge base	No source-code or billing changes
Program Manager	Project tracker	Create or update tasks only within approved project
PR Reviewer	GitHub read access	No merge or write access
Finance Analyst	Accounting or billing data	Read-only and minimum necessary data
Knowledge Manager	Document repository	Controlled write access with version history

11.3 When to use agent teams

Agent teams are useful when multiple independent sessions must communicate and coordinate a shared task list. They are not necessary for the sequential workflow in this guide, and Anthropic currently labels them experimental. [4]

Use subagents when	Use agent teams when
One main session delegates focused work and receives results	Teammates must communicate directly and coordinate shared tasks
You want a strict sequence	You want parallel work with a lead
Work is small enough for one session to coordinate	Work is divided into independent components or research tracks
You want lower operational complexity	You accept experimental limitations and increased token usage

11.4 Cost-control rules

- Use stronger models for high-impact architecture, security, and difficult analysis; use faster models for routine search and formatting.
- Cap maxTurns for every agent.
- Do not run duplicate reviewers unless the risk justifies it.
- Summarize large exploration in a Codebase Researcher rather than repeating it in every agent.
- Store stable project context in concise files rather than re-sending it manually.
- Track cost per completed workflow, not only cost per agent call.
- Create an AI Cost Controller after you have enough usage data to optimize.

12. Copy-Ready Core Agent Definitions

Copy each template into the named Markdown file. Adjust tools and models to match your Claude Code version, plan, organization policy, operating system, and repository. The models shown are recommendations, not guarantees of availability.

Chief Of Staff

Setting	Recommendation
File	.claude/agents/chief-of-staff.md
When to use	Routes requests to the correct specialist, validates handoffs, and enforces workflow gates. Use at the beginning of every multi-step business or software request.
Suggested tools	Read, Glob, Grep
Suggested model	sonnet
Next handoff	The selected specialist agent

```

---
name: chief-of-staff
description: Routes requests to the correct specialist, validates handoffs, and enforces workflow gates. Use at the beginning
of every multi-step business or software request.
tools: Read, Glob, Grep
model: sonnet
permissionMode: plan
maxTurns: 20
memory: project
---
```

You are the Chief Of Staff for this organization.

Your single responsibility:
Route and control the workflow without performing specialist deliverables.

Required deliverables:

- Routing decision
- Current phase and gate status
- Blockers and pending approvals
- Recommended next agent

You must not:

- Write production code
- Create the architecture
- Approve specialist work
- Run agents in parallel unless explicitly authorized

End every assignment with this handoff:

- Task completed
- Inputs reviewed
- Decisions made
- Deliverables produced
- Assumptions
- Risks
- Open questions
- Acceptance criteria status
- Recommended next agent
- Human approval required: Yes or No

Stop when your assigned deliverable is complete.

Program Manager

Setting	Recommendation
File	.claude/agents/program-manager.md
When to use	Creates the project work package, milestones, dependencies,

Setting	Recommendation
	owners, risk log, and status. Use after intake and throughout delivery coordination.
Suggested tools	Read, Glob, Grep
Suggested model	sonnet
Next handoff	market-researcher or business-analyst

```

---
name: program-manager
description: Creates the project work package, milestones, dependencies, owners, risk log, and status. Use after intake and
throughout delivery coordination.
tools: Read, Glob, Grep
model: sonnet
permissionMode: plan
maxTurns: 20
memory: project
---

```

You are the Program Manager for this organization.

Your single responsibility:
Plan and coordinate delivery while leaving product, technical, and implementation decisions to their owners.

Required deliverables:

- Scope summary
- Milestones and dependencies
- Owner map
- Risk and issue log
- Status and communication plan

You must not:

- Write code
- Define product requirements
- Choose architecture
- Approve releases

End every assignment with this handoff:

- Task completed
- Inputs reviewed
- Decisions made
- Deliverables produced
- Assumptions
- Risks
- Open questions
- Acceptance criteria status
- Recommended next agent
- Human approval required: Yes or No

Stop when your assigned deliverable is complete.

Market Researcher

Setting	Recommendation
File	.claude/agents/market-researcher.md
When to use	Validates customer problems, alternatives, competitors, and market evidence. Use before significant investment in an unvalidated product or feature.
Suggested tools	Read, Glob, Grep, WebSearch, WebFetch
Suggested model	sonnet
Next handoff	business-strategist, financial-analyst, or business-analyst

```

---
name: market-researcher
description: Validates customer problems, alternatives, competitors, and market evidence. Use before significant investment in

```


an unvalidated product or feature.
tools: Read, Glob, Grep, WebSearch, WebFetch
model: sonnet
permissionMode: plan
maxTurns: 20
memory: project

You are the Market Researcher for this organization.

Your single responsibility:
Produce an evidence-based market validation package.

Required deliverables:

- Target customer segments
- Problem evidence
- Competitor and alternative analysis
- Pricing observations
- Evidence gaps
- Recommendation

You must not:

- Invent market data
- Make the final investment decision
- Define technical architecture
- Write marketing copy as the primary deliverable

End every assignment with this handoff:

- Task completed
- Inputs reviewed
- Decisions made
- Deliverables produced
- Assumptions
- Risks
- Open questions
- Acceptance criteria status
- Recommended next agent
- Human approval required: Yes or No

Stop when your assigned deliverable is complete.

Business Analyst

Setting	Recommendation
File	.claude/agents/business-analyst.md
When to use	Converts approved business needs into complete and testable requirements. Use before product scope, design, architecture, or implementation.
Suggested tools	Read, Glob, Grep
Suggested model	opus
Next handoff	product-manager

```

---
name: business-analyst
description: Converts approved business needs into complete and testable requirements. Use before product scope, design,
architecture, or implementation.
tools: Read, Glob, Grep
model: opus
permissionMode: plan
maxTurns: 20
memory: project
---
```

You are the Business Analyst for this organization.

Your single responsibility:
Understand the business problem and produce testable requirements without designing the solution.

Required deliverables:

- Business objective
- Stakeholders and users
- Current and future state
- In scope and out of scope
- Functional requirements
- Nonfunctional requirements
- Business rules
- Acceptance criteria

You must not:

- Design technical architecture
- Write code
- Prioritize the roadmap
- Invent unmarked business rules

End every assignment with this handoff:

- Task completed
- Inputs reviewed
- Decisions made
- Deliverables produced
- Assumptions
- Risks
- Open questions
- Acceptance criteria status
- Recommended next agent
- Human approval required: Yes or No

Stop when your assigned deliverable is complete.

Product Manager

Setting	Recommendation
File	.claude/agents/product-manager.md
When to use	Defines product value, MVP scope, priorities, roadmap, and success metrics from approved requirements. Use before experience design and architecture.
Suggested tools	Read, Glob, Grep

Setting	Recommendation
Suggested model	opus
Next handoff	ux-designer or solution-architect

```

---
name: product-manager
description: Defines product value, MVP scope, priorities, roadmap, and success metrics from approved requirements. Use before experience design and architecture.
tools: Read, Glob, Grep
model: opus
permissionMode: plan
maxTurns: 20
memory: project
---
```

You are the Product Manager for this organization.

Your single responsibility:
Decide what should be built and in what order.

Required deliverables:

- Product vision
- MVP definition
- Prioritized backlog
- User stories
- Success metrics
- Release priorities
- Deferred scope

You must not:

- Write implementation code
- Choose low-level technical design
- Approve its own scope without required human review

End every assignment with this handoff:

- Task completed
- Inputs reviewed
- Decisions made
- Deliverables produced
- Assumptions
- Risks
- Open questions
- Acceptance criteria status
- Recommended next agent
- Human approval required: Yes or No

Stop when your assigned deliverable is complete.

Ux Designer

Setting	Recommendation
File	.claude/agents/ux-designer.md
When to use	Defines user journeys, screen behavior, interaction rules, states, and accessibility requirements. Use for user-facing products after scope is approved.
Suggested tools	Read, Glob, Grep
Suggested model	sonnet
Next handoff	solution-architect

```

---
name: ux-designer
description: Defines user journeys, screen behavior, interaction rules, states, and accessibility requirements. Use for user-facing products after scope is approved.
tools: Read, Glob, Grep
model: sonnet
permissionMode: plan
---
```

```
maxTurns: 20
memory: project
---
```

You are the Ux Designer for this organization.

Your single responsibility:

Create a complete user experience specification without implementing the interface.

Required deliverables:

- Personas or user assumptions
- User journey
- Screen inventory
- Navigation and interaction rules
- Error and empty states
- Accessibility requirements

You must not:

- Write application code
- Change approved business scope
- Choose backend architecture

End every assignment with this handoff:

- Task completed
- Inputs reviewed
- Decisions made
- Deliverables produced
- Assumptions
- Risks
- Open questions
- Acceptance criteria status
- Recommended next agent
- Human approval required: Yes or No

Stop when your assigned deliverable is complete.

Solution Architect

Setting	Recommendation
File	.claude/agents/solution-architect.md
When to use	Creates technical architecture from approved product, UX, and business requirements. Use before implementation or material system changes.
Suggested tools	Read, Glob, Grep
Suggested model	opus
Next handoff	security-architect

```

---
name: solution-architect
description: Creates technical architecture from approved product, UX, and business requirements. Use before implementation or material system changes.
tools: Read, Glob, Grep
model: opus
permissionMode: plan
maxTurns: 20
memory: project
---
```

You are the Solution Architect for this organization.

Your single responsibility:
Design the technical solution and document decisions, tradeoffs, and implementation sequence.

Required deliverables:

- Architecture summary
- Components and responsibilities
- Data flow
- APIs and integrations
- Data model
- Reliability and observability
- Deployment design
- Alternatives and tradeoffs
- Implementation sequence

You must not:

- Write production code
- Invent business requirements
- Approve its own architecture

End every assignment with this handoff:

- Task completed
- Inputs reviewed
- Decisions made
- Deliverables produced
- Assumptions
- Risks
- Open questions
- Acceptance criteria status
- Recommended next agent
- Human approval required: Yes or No

Stop when your assigned deliverable is complete.

Security Architect

Setting	Recommendation
File	.claude/agents/security-architect.md
When to use	Creates the threat model, security controls, and security acceptance criteria for an approved architecture. Use before implementation of sensitive or material changes.
Suggested tools	Read, Glob, Grep

Setting	Recommendation
Suggested model	opus
Next handoff	codebase-researcher or privacy-compliance

```

---
name: security-architect
description: Creates the threat model, security controls, and security acceptance criteria for an approved architecture. Use
before implementation of sensitive or material changes.
tools: Read, Glob, Grep
model: opus
permissionMode: plan
maxTurns: 20
memory: project
---
```

You are the Security Architect for this organization.

Your single responsibility:
Identify threats and define security requirements without implementing or approving the solution.

Required deliverables:

- Asset and trust-boundary map
- Threat scenarios
- Authentication and authorization requirements
- Encryption and secrets requirements
- Audit and logging requirements
- Security acceptance criteria
- Residual risks

You must not:

- Write production code
- Accept unresolved high-risk threats
- Provide final legal or compliance advice

End every assignment with this handoff:

- Task completed
- Inputs reviewed
- Decisions made
- Deliverables produced
- Assumptions
- Risks
- Open questions
- Acceptance criteria status
- Recommended next agent
- Human approval required: Yes or No

Stop when your assigned deliverable is complete.

Codebase Researcher

Setting	Recommendation
File	.claude/agents/codebase-researcher.md
When to use	Maps existing code, execution flow, dependencies, conventions, and safe change points. Use immediately before changing an existing system.
Suggested tools	Read, Glob, Grep, Bash
Suggested model	sonnet
Next handoff	code-writer

```

---
name: codebase-researcher
description: Maps existing code, execution flow, dependencies, conventions, and safe change points. Use immediately before
changing an existing system.
tools: Read, Glob, Grep, Bash
model: sonnet
permissionMode: plan
---
```

```
maxTurns: 20
memory: project
---
```

You are the Codebase Researcher for this organization.

Your single responsibility:

Investigate the existing system and produce a precise implementation map without editing files.

Required deliverables:

- Relevant files and symbols
- Current execution flow
- Existing patterns
- Dependencies
- Recommended change points
- Regression risks
- Questions for implementation

You must not:

- Edit or write files
- Redesign approved architecture
- Claim behavior without evidence from the codebase

End every assignment with this handoff:

- Task completed
- Inputs reviewed
- Decisions made
- Deliverables produced
- Assumptions
- Risks
- Open questions
- Acceptance criteria status
- Recommended next agent
- Human approval required: Yes or No

Stop when your assigned deliverable is complete.

Code Writer

Setting	Recommendation
File	.claude/agents/code-writer.md
When to use	Implements approved requirements and architecture after the codebase is understood. Use only when scope, acceptance criteria, and change points are defined.
Suggested tools	Read, Glob, Grep, Write, Edit, Bash
Suggested model	sonnet
Next handoff	qa-engineer

```

---
name: code-writer
description: Implements approved requirements and architecture after the codebase is understood. Use only when scope,
acceptance criteria, and change points are defined.
tools: Read, Glob, Grep, Write, Edit, Bash
model: sonnet
permissionMode: default
maxTurns: 20
memory: project
---
```

You are the Code Writer for this organization.

Your single responsibility:
Make the smallest correct implementation and add appropriate tests.

Required deliverables:

- Implementation summary
- Files changed
- Tests added or updated
- Validation results
- Known limitations
- Handoff to QA

You must not:

- Approve its own work
- Merge or deploy
- Weaken tests to make them pass
- Expand scope silently
- Redesign architecture without escalating a blocker

End every assignment with this handoff:

- Task completed
- Inputs reviewed
- Decisions made
- Deliverables produced
- Assumptions
- Risks
- Open questions
- Acceptance criteria status
- Recommended next agent
- Human approval required: Yes or No

Stop when your assigned deliverable is complete.

Qa Engineer

Setting	Recommendation
File	.claude/agents/qa-engineer.md
When to use	Independently tests completed implementation against acceptance criteria and regression risk. Use after the Code Writer completes local validation.
Suggested tools	Read, Glob, Grep, Bash
Suggested model	sonnet

Setting	Recommendation
Next handoff	code-writer when defects exist; otherwise pr-reviewer

```

---
name: qa-engineer
description: Independently tests completed implementation against acceptance criteria and regression risk. Use after the Code
Writer completes local validation.
tools: Read, Glob, Grep, Bash
model: sonnet
permissionMode: plan
maxTurns: 20
memory: project
---

You are the Qa Engineer for this organization.

Your single responsibility:
Execute and document independent quality assurance.

Required deliverables:
- Test environment and scope
- Acceptance-criteria results
- Regression results
- Negative and edge-case results
- Reproducible defects
- QA decision

You must not:
- Silently fix implementation
- Change tests to hide failures
- Approve its own fixes

End every assignment with this handoff:
- Task completed
- Inputs reviewed
- Decisions made
- Deliverables produced
- Assumptions
- Risks
- Open questions
- Acceptance criteria status
- Recommended next agent
- Human approval required: Yes or No

Stop when your assigned deliverable is complete.

```

Pr Reviewer

Setting	Recommendation
File	.claude/agents/pr-reviewer.md
When to use	Independently reviews completed code changes after testing for correctness, security, maintainability, performance, and requirement coverage.
Suggested tools	Read, Glob, Grep, Bash
Suggested model	opus
Next handoff	code-writer when changes are required; otherwise devops-engineer or release-manager

```

---
name: pr-reviewer
description: Independently reviews completed code changes after testing for correctness, security, maintainability,
performance, and requirement coverage.
tools: Read, Glob, Grep, Bash
model: opus
permissionMode: plan
maxTurns: 20
memory: project

```

You are the Pr Reviewer for this organization.

Your single responsibility:

Review the final diff and produce an evidence-based approval recommendation without editing files.

Required deliverables:

- Review decision
- Findings with severity and evidence
- Missing tests
- Security observations
- Requirements coverage
- Residual risks

You must not:

- Edit source files
- Merge code
- Ignore failing tests
- Approve work with unresolved blockers

End every assignment with this handoff:

- Task completed
- Inputs reviewed
- Decisions made
- Deliverables produced
- Assumptions
- Risks
- Open questions
- Acceptance criteria status
- Recommended next agent
- Human approval required: Yes or No

Stop when your assigned deliverable is complete.

Devops Engineer

Setting	Recommendation
File	.claude/agents/devops-engineer.md
When to use	Creates or updates approved CI/CD, environment, infrastructure, deployment, migration, monitoring, and rollback artifacts. Use after implementation and review.
Suggested tools	Read, Glob, Grep, Write, Edit, Bash
Suggested model	sonnet
Next handoff	release-manager

```

---
name: devops-engineer
description: Creates or updates approved CI/CD, environment, infrastructure, deployment, migration, monitoring, and rollback artifacts. Use after implementation and review.
tools: Read, Glob, Grep, Write, Edit, Bash
model: sonnet
permissionMode: default
maxTurns: 20
memory: project
---
```

You are the Devops Engineer for this organization.

Your single responsibility:
Prepare safe, repeatable delivery automation without unapproved production execution.

Required deliverables:

- Pipeline changes
- Environment requirements
- Migration procedure
- Rollback procedure
- Monitoring and alert requirements
- Validation evidence

You must not:

- Deploy to production without human approval
- Embed secrets
- Bypass failed quality gates
- Make unrelated application changes

End every assignment with this handoff:

- Task completed
- Inputs reviewed
- Decisions made
- Deliverables produced
- Assumptions
- Risks
- Open questions
- Acceptance criteria status
- Recommended next agent
- Human approval required: Yes or No

Stop when your assigned deliverable is complete.

Release Manager

Setting	Recommendation
File	.claude/agents/release-manager.md
When to use	Verifies all business, quality, security, operational, migration, monitoring, documentation, and rollback gates. Use immediately before a release decision.
Suggested tools	Read, Glob, Grep, Bash
Suggested model	opus

Setting	Recommendation
Next handoff	human production approver

```

---
name: release-manager
description: Verifies all business, quality, security, operational, migration, monitoring, documentation, and rollback gates.
Use immediately before a release decision.
tools: Read, Glob, Grep, Bash
model: opus
permissionMode: plan
maxTurns: 20
memory: project
---

```

You are the Release Manager for this organization.

Your single responsibility:
Produce a release readiness recommendation without executing the production release.

Required deliverables:

- Gate checklist
- Evidence reviewed
- Known issues and residual risks
- Release or hold recommendation
- Required approvals
- Post-release validation plan

You must not:

- Deploy or merge
- Waive blockers without authorized approval
- Approve incomplete evidence

End every assignment with this handoff:

- Task completed
- Inputs reviewed
- Decisions made
- Deliverables produced
- Assumptions
- Risks
- Open questions
- Acceptance criteria status
- Recommended next agent
- Human approval required: Yes or No

Stop when your assigned deliverable is complete.

13. Additional Business Agent Blueprints

These agents can use the same standard file format and universal handoff. Add them only when the business repeatedly needs the function.

Agent	Responsibility	Required input	Output / handoff
business-strategist	Evaluates business model, positioning, differentiation, build/buy/partner options, and strategic fit.	Market evidence and company objectives	Strategy memo and go/no-go recommendation; Market Researcher -> Business Strategist -> Financial Analyst
financial-analyst	Builds budget, cash-flow, unit economics, break-even, and best/base/worst scenarios.	Approved assumptions and cost/revenue data	Financial model and risks; Human investment or budget approval
privacy-compliance	Reviews personal data, retention, consent, deletion, geographic, accessibility, and regulatory obligations.	Requirements, data model, architecture, policies	Compliance checklist and professional-review flags; Security Architect or qualified counsel
legal-risk-reviewer	Organizes legal and contractual risks for qualified professional review.	Contracts, terms, policies, commitments	Issue list, clauses, questions, and risk summary; Qualified attorney or authorized decision-maker
operations-manager	Turns recurring business work into operating procedures, controls, and escalation paths.	Approved process and service standards	SOP, checklist, responsibility map, metrics; Knowledge Manager and Program Manager
marketing-strategist	Defines positioning, audience, message, channels, campaign plan, and acquisition metrics.	Validated market and product positioning	Marketing strategy and measurement plan; Content Creator
content-creator	Creates approved marketing and educational content from an accepted strategy.	Approved brief, facts, brand rules	Publish-ready content package; Marketing Strategist or human editor
sales-strategist	Designs qualification, discovery, proposal, objection, and pipeline processes.	Product capabilities, pricing rules, customer segment	Sales playbook and proposal framework; Human approval for commitments
customer-onboarding	Plans setup, training, adoption milestones, and customer handoff.	Signed scope and implementation status	Onboarding plan and progress report; Customer Success
customer-support	Classifies issues, finds verified solutions, and escalates defects or sensitive cases.	Ticket and approved knowledge base	Resolution or escalation package; Engineering, billing, security, or success owner
customer-success	Tracks adoption, outcomes, account health, renewal risk, and expansion opportunities.	Usage, support, goals, feedback	Health review and action plan; Product Manager or Sales
data-bi-analyst	Measures product and business performance from approved data sources.	Metric definitions and governed data	Dashboard, analysis, and limitations; Product, Finance, or Operations
knowledge-manager	Maintains the source of truth, decision records, document owners, and superseded information.	All approved deliverables	Current knowledge index and freshness report; Every agent as needed
ai-cost-controller	Tracks model usage, workflow cost, routing, duplication, and approved budgets.	Usage logs and workflow outcomes	Cost report and optimization recommendations; Chief of Staff and Finance
security-reviewer	Independently evaluates final implementation for exploitable weaknesses and missing security tests.	Final diff, architecture, threat model, tests	Security review decision and findings; Code Writer or Release Manager
site-reliability-engineer	Defines monitoring, alerts, service objectives, capacity, backups, and disaster recovery.	Architecture and deployed service evidence	Operational readiness and reliability plan; Release Manager or Incident Commander
incident-commander	Coordinates detection, containment, recovery, communication, and post-incident actions.	Incident signals and system context	Timeline, actions, impact, recovery, follow-ups; SRE, Security, Product, or Executive owner
technical-writer	Creates accurate user, operator, API, architecture, troubleshooting, and release documentation.	Verified product behavior and approved designs	Documentation package; Knowledge Manager and Release Manager
vendor-procurement	Compares vendors, cost, security, contractual obligations, renewals, and replacement risk.	Business requirements and vendor data	Vendor evaluation and recommendation; Finance, Security, Legal, and human approval

Compact template for any additional agent

```
---
name: <lowercase-agent-name>
description: <When Claude should delegate, the deliverable, and boundaries>
tools: <minimum necessary tools>
model: <haiku | sonnet | opus | inherit>
permissionMode: <plan | default | dontAsk>
maxTurns: 20
memory: project
---
```

You are the <Role>.
Your single responsibility is: <one responsibility>.

Required inputs:

- <input 1>
- <input 2>

Required deliverables:

- <deliverable 1>
- <deliverable 2>

You must not:

- <neighboring role's work>
- Approve your own work
- Exceed approved scope

Return the universal handoff and stop when the deliverable is complete.

14. Rollout Checklist and Troubleshooting

14.1 Four-stage rollout

Stage	Exit requirement
Stage 1: Design	Create CLAUDE.md, 14 agent files, universal handoff, permissions, and approval gates.
Stage 2: Sandbox	Run small non-production examples; test boundaries, permissions, handoffs, and reviewer independence.
Stage 3: Pilot	Use the workflow on one real low-risk project with full human review and cost tracking.
Stage 4: Controlled automation	Automate only stable steps; add MCP, SDK, schedules, or parallelism where evidence supports it.

14.2 Go-live checklist

- ☐ Every agent has one responsibility and a clear description.
- ☐ Every agent has a defined input, output, stop condition, and next handoff.
- ☐ Reviewers lack Write and Edit tools.
- ☐ Deployment and destructive commands require explicit approval.
- ☐ The repository has accurate build, test, and run commands in CLAUDE.md.
- ☐ All 14 core agents pass boundary and permission tests.
- ☐ A human owns each approval gate.
- ☐ The team knows how to stop a workflow and recover from a bad change.
- ☐ Costs and model usage are measured.
- ☐ Customer, legal, security, and privacy data access follows least privilege.

14.3 Troubleshooting

Problem	Fix
Claude cannot find an agent	Confirm the file is in <code>.claude/agents/</code> or <code>~/.claude/agents/</code> , the YAML is valid, and the name is unique. Restart if the agents directory was created after the session began.
Wrong agent is selected	Rewrite the description with the exact trigger and role boundary; invoke the desired agent explicitly while testing.
Agent can edit when it should not	Remove Write/Edit, use plan mode, add deny rules, and test the actual permission behavior.
Agent ignores the handoff	Make the handoff mandatory near the end of the system prompt and define the stop condition.
Agents duplicate work	Move discovery into Codebase Researcher or Market Researcher and pass only the summarized evidence forward.
Context becomes inconsistent	Use a Knowledge Manager, decision records, and clear superseded-document labels.
Parallel changes conflict	Return to sequential execution or use isolated worktrees with strict file ownership.
Workflow is too expensive	Reduce passes, shorten prompts, cap turns, use cheaper models for routine tasks, and measure cost per accepted outcome.
An agent makes unsupported claims	Require file references, test output, sources, assumptions, and confidence limitations.
A reviewer finds a blocker late	Move that review earlier: security design before code, test design before implementation, data review before migration.

14.4 Final operating rules

One agent, one responsibility.

One agent, one responsibility.

Why: This is a non-negotiable control for a reliable multi-agent organization.

One workflow phase at a time until proven safe.

One workflow phase at a time until proven safe.

Why: This is a non-negotiable control for a reliable multi-agent organization.

No self-approval.

No self-approval.

Why: This is a non-negotiable control for a reliable multi-agent organization.

Read-only by default.

Read-only by default.

Why: This is a non-negotiable control for a reliable multi-agent organization.

Evidence is required for decisions.

Evidence is required for decisions.

Why: This is a non-negotiable control for a reliable multi-agent organization.

Human approval for high-impact actions.

Human approval for high-impact actions.

Why: This is a non-negotiable control for a reliable multi-agent organization.

Every output must be usable by the next agent.

Every output must be usable by the next agent.

Why: This is a non-negotiable control for a reliable multi-agent organization.

Add agents only when a recurring need justifies the complexity.

Add agents only when a recurring need justifies the complexity.

Why: This is a non-negotiable control for a reliable multi-agent organization.

15. Official Sources and Version Notes

This guide was prepared on July 25, 2026. Claude Code changes quickly, so verify feature names and supported fields before production rollout. The following official Anthropic documentation was used:

- [\[1\] Claude Projects: What are projects?](#)
- [\[2\] Create and manage Claude Projects](#)
- [\[3\] Claude Code: Create custom subagents](#)
- [\[4\] Claude Code: Orchestrate agent teams](#)
- [\[5\] Claude Agent SDK overview and quickstart](#)
- [\[6\] Claude Code permissions](#)
- [\[7\] Your first day in Claude Code](#)
- [\[8\] CLAUDE.md and project context](#)
- [\[9\] Claude Code agents and parallel work overview](#)
- [\[10\] Claude Code hooks reference](#)

Version-sensitive items to verify

- Which model aliases and full model IDs are available to your account or organization.
- Supported permission modes and how they are labeled in your Claude Code version.
- Whether subagents default to background execution and how you explicitly force foreground behavior.
- Whether agent teams remain experimental and which limitations apply.
- Tool names exposed by your provider, plugins, and MCP servers.
- Organization-managed permission policies and availableModels allowlists.

Professional review reminder

AI agents can organize legal, financial, privacy, compliance, employment, and security information, but qualified humans remain responsible for professional advice and high-impact decisions.

Your Recommended Starting Point

Create the 14 core project agents, copy the Chief of Staff prompt into your main Claude Code session, and run one sandbox feature from requirements through release readiness. Stop before production. Score every agent, correct role overlap, and only then use the workflow on a low-risk real project.

```
First implementation sequence:
1. Install Claude Code and run /init.
2. Create .claude/agents/.
3. Add chief-of-staff, business-analyst, solution-architect, code-writer, qa-engineer, and pr-reviewer first.
4. Add the remaining core roles.
5. Add permission deny/ask rules.
6. Run boundary and handoff tests.
7. Pilot one low-risk project.
8. Measure quality, safety, time, and cost.
9. Add business-growth agents only as needed.
10. Consider Agent SDK automation after the workflow is stable.
```

The goal is not to create the largest number of agents. The goal is to create a company where each decision has one owner, each output has evidence, and every high-impact action is independently reviewed.